

Практична робота №2

Взаємодія вебзастосунку з користувачем: форми, валідація, локальне сховище браузера

Тривалість: 90 хв

Формат: комп'ютерний практикум

Мета роботи

Навчитися обробляти дані HTML-форм у PHP через суперглобальні масиви `$_GET` і `$_POST`; освоїти серверну валідацію введених даних (обов'язковість, тип, діапазон, формат) і повторне відображення форми з повідомленнями про помилки; додати клієнтську валідацію засобами HTML5 та JavaScript; навчитися зберігати й відновлювати стан інтерфейсу за допомогою `localStorage/sessionStorage` браузера.

Необхідні знання й інструменти

Знання: базовий синтаксис PHP - змінні, масиви, умовні оператори, цикли, функції (практикум №1); структура HTML-форми (тег `<form>`, елементи `<input>`, `<select>`, `<textarea>`, атрибути `method`, `action`, `name`); бажані базові поняття JavaScript (змінні, функції, обробники подій) - короткий мінімум наведено в теоретичному вступі.

Інструменти: редактор коду (наприклад, VS Code); локальний вебсервер із PHP (Apache/PHP через XAMPP/OpenServer або вбудований сервер ``php -S localhost:8000``); браузер із інструментами розробника (вкладка Application/Storage для перегляду `localStorage`); доступ до файлової системи проєкту.

Короткий теоретичний вступ

HTML-форма (`<form method="post" action="process.php">`) надсилає введені користувачем значення на сервер; кожне поле форми потрапляє в PHP-скрипт під ключем, що відповідає його атрибуту `name`. Дані форми з методом POST доступні через суперглобальний масив `$_POST`, з методом GET - через `$_GET` (значення при цьому додаються до URL-адреси рядком запиту після `?`, тому GET не підходить для чутливих даних чи великих обсягів).

```
<?php
$errors = [];
$title  = trim($_POST['title'] ?? '');
$year   = $_POST['year'] ?? '';

if ($title === '') {
    $errors['title'] = 'Назва обов'язкова';
```

```

}
if (!is_numeric($year) || $year < 1450 || $year > (int) date('Y')) {
    $errors['year'] = 'Рік має бути числом у допустимому діапазоні';
}

if (empty($errors)) {
    echo 'Дані прийнято: ' . htmlspecialchars($title);
}

```

Серверна валідація обов'язкова, навіть якщо форма вже має клієнтську перевірку: HTML5-атрибути (required, min, max, pattern, type="email") і JavaScript працюють лише в браузері, а запит на сервер можна надіслати напряму (curl, Postman, вимкнений JavaScript), обійшовши будь-яку клієнтську перевірку. Тому кожне поле обов'язково перевіряють ще раз у PHP-кодї, а введені користувачем дані, які повертаються на сторінку (наприклад, у повідомленні про помилку чи для повторного заповнення форми), виводять через htmlspecialchars() - це запобігає найпростішим XSS-атакам.

localStorage і sessionStorage - це JavaScript API для збереження пар «ключ - значення» безпосередньо в браузері, без звернення до сервера (на відміну від cookie, які автоматично надсилаються з кожним запитом). localStorage зберігає дані без терміну дії, доки їх явно не видалять; sessionStorage - лише на час роботи з вкладкою. Обидва сховища працюють лише з рядками, тому об'єкти й масиви перед записом перетворюють у рядок через JSON.stringify(), а при читанні відновлюють через JSON.parse().

```

const input = document.getElementById('title');
input.value = localStorage.getItem('draft_title') || '';

input.addEventListener('input', () => {
    localStorage.setItem('draft_title', input.value);
});

```

- **\$_POST / \$_GET** - суперглобальні масиви PHP з даними форми: \$_POST - для методу POST (у тілі запиту, не видно в адресному рядку), \$_GET - для методу GET (у рядку параметрів URL після ?).
- **Серверна валідація** - перевірка отриманих даних безпосередньо в PHP-кодї на сервері; на відміну від клієнтської, її не можна обійти, надіславши запит напряму, тому вона обов'язкова для будь-якої форми.
- **htmlspecialchars()** - функція, що екранує спецсимволи HTML (<, >, &, лапки) перед виведенням даних, уведених користувачем - базовий захист від XSS.
- **HTML5-атрибути валідації** - required, min, max, pattern, type="email" тощо - браузер не дає надіслати форму, якщо поле не відповідає умові; зручно для користувача, але не замінює серверну перевірку.
- **localStorage / sessionStorage** - JavaScript API для збереження рядкових пар «ключ - значення» у браузері: localStorage - без терміну дії, sessionStorage - на час вкладки; дані не надсилаються на сервер автоматично (на відміну від cookie).
- **JSON.stringify() / JSON.parse()** - перетворення об'єкта чи масиву JavaScript у рядок для збереження в localStorage і, відповідно, відновлення рядка назад в об'єкт/масив при читанні.

Хід роботи

Структура завдання спільна для всіх варіантів - відрізняється лише предметна область, набір полів форми і сценарій localStorage (див. розділ '20 варіантів завдання'). Виконуйте кроки послідовно.

Крок 1. Підготувати структуру проєкту.

Створіть папку практикуму (наприклад, practicum02/) з файлом form.php, що міститиме і HTML-форму, і PHP-обробник (простіше для навчального завдання, ніж окремий process.php).

Крок 2. Створити HTML-форму.

Додайте форму (method="post", action="form.php") із полями свого варіанта: текстові поля, число, за потреби select або checkbox. Для кожного поля додайте відповідні HTML5-атрибути валідації (required, min/max, pattern тощо) згідно з правилами варіанта.

Крок 3. Реалізувати серверну обробку й валідацію.

На початку файлу перевірте, чи форму вже надіслано (\$_SERVER['REQUEST_METHOD'] === 'POST'). Отримайте значення полів через \$_POST, для кожного поля виконайте перевірку згідно з правилами свого варіанта (обов'язковість, тип/діапазон, формат через preg_match за потреби) і накопичте масив помилок.

Крок 4. Показати результат обробки.

Якщо масив помилок не порожній - повторно виведіть форму, показавши повідомлення про кожну помилку і підставивши вже введені користувачем значення в поля (щоб не змушувати вводити все заново); значення обов'язково виводьте через htmlspecialchars(). Якщо помилок немає - виведіть підтвердження з прийнятими даними.

Крок 5. Додати клієнтську JavaScript-валідацію.

Додайте невеликий скрипт, що перед відправкою форми перевіряє хоча б одне поле (наприклад, довжину чи формат) і показує повідомлення без перезавантаження сторінки (addEventListener('submit', ...) з event.preventDefault() у разі помилки).

Крок 6. Реалізувати сценарій localStorage/sessionStorage.

Відповідно до свого варіанта реалізуйте на JavaScript збереження потрібних даних у localStorage чи sessionStorage та їх відновлення після перезавантаження сторінки (значення читається й підставляється в поле/елемент інтерфейсу одразу після завантаження сторінки).

Крок 7. Перевірити роботу.

Перевірте форму з коректними даними, з некоректними даними для кожного правила валідації (порожнє поле, значення поза діапазоном, неправильний формат) і роботу localStorage - значення повинні зберігатися і відновлюватися після перезавантаження сторінки.

20 варіантів завдання

Наскрізні варіанти - один і той самий номер відповідає одному домену протягом усього курсу (всі 8 практикумів). Виберіть свій варіант (за номером у списку групи чи вказівкою викладача) і виконуйте завдання для нього. У кожному рядку: поля форми, правила серверної валідації та сценарій використання localStorage/sessionStorage.

№ / Домен	Зміст варіанта
1. Бібліотека (каталог книг, видача/повернення)	Форма: додавання книги - title, author, year, isbn. Валідація: title і author обов'язкові; year - число в діапазоні 1450–поточний рік; isbn - формат із 10 або 13 цифр (перевірка через preg_match). localStorage: зберігати чернетку форми (щоб не втратити введені дані при випадковому закритті вкладки чи перезавантаженні сторінки).
2. Інтернет-магазин (каталог товарів, кошик)	Форма: додавання товару - name, price, sku, stock. Валідація: price - число більше 0; stock - ціле невід'ємне число; sku - формат «ABC-123» (літери-дефіс-цифри). localStorage: запам'ятовувати останню обрану категорію фільтра каталогу.
3. Блог/CMS для статей	Форма: нова стаття - title, content, tags. Валідація: title - від 5 до 100 символів; content - не менше 20 символів. localStorage: автозбереження чернетки статті (наприклад, при кожному введенні тексту).
4. Фітнес-трекер (тренування, вправи)	Форма: нове тренування - type, durationMin, caloriesBurned, date. Валідація: durationMin - число від 1 до 300; date - коректна дата, не пізніша за сьогодні. localStorage: запам'ятати останній обраний тип тренування для наступного відкриття форми.
5. Кінотека (фільми, рецензії)	Форма: рецензія на фільм - movieTitle, rating, reviewText. Валідація: rating - ціле число від 1 до 10; reviewText - не менше 10 символів. localStorage: зберігати чернетку тексту рецензії.
6. Кулінарна книга (рецепти, інгредієнти)	Форма: новий рецепт - title, ingredients, cookTimeMin. Валідація: cookTimeMin - число більше 0; ingredients - непорожній перелік (мінімум один інгредієнт). localStorage: запам'ятати обрану одиницю виміру (грами/унції) для полів інгредієнтів.
7. Трекер завдань (to-do list, проекти)	Форма: нове завдання - title, dueDate, priority (select). Валідація: title обов'язкове; dueDate - коректна дата; priority - одне з дозволених значень списку. localStorage: зберігати чернетку незавершеного завдання до моменту надсилання форми.
8. Система бронювання (готель/зали)	Форма: бронювання - guestName, roomNumber, checkIn, checkOut. Валідація: checkOut пізніше за checkIn; guestName - лише літери й пробіли (preg_match). localStorage: запам'ятати останні обрані дати заїзду й виїзду для

	наступного бронювання.
9. Форум/дошка обговорень	Форма: нова тема - title, author, message. Валідація: message - не менше 10 символів; title - без заборонених спецсимволів (перевірка через preg_match). localStorage: зберігати чернетку повідомлення форуму.
10. Каталог вакансій (job board)	Форма: нова вакансія - title, company, salaryFrom, salaryTo, remote (checkbox). Валідація: salaryFrom і salaryTo - числа, salaryTo >= salaryFrom. localStorage: запам'ятати стан прапорця «тільки віддалені» у фільтрі вакансій.
11. Система обліку студентів/оцінок	Форма: додавання оцінки - studentName, subject, grade. Валідація: studentName обов'язкове; grade - число в межах допустимої шкали (наприклад, 0–100). localStorage: запам'ятати останню обрану групу для наступного відкриття форми.
12. Онлайн-опитування/анкети	Форма: нове питання - questionText, type (select), options. Валідація: якщо type == 'multiple' - options повинні містити щонайменше два варіанти. localStorage: зберігати чернетку форми створення питання.
13. Трекер особистих витрат (фінанси)	Форма: нова транзакція - amount, category (select), date. Валідація: amount - число більше 0; category обов'язкова; date - коректна дата. localStorage: запам'ятати останню обрану категорію за замовчуванням.
14. Каталог музики/плейлисти	Форма: додавання треку - title, artist, durationSec. Валідація: durationSec - число більше 0; title і artist обов'язкові. localStorage: запам'ятати назву останнього відкритого плейлиста.
15. Система бронювання квитків (кіно/події)	Форма: бронювання квитка - eventTitle, seatsCount, buyerEmail. Валідація: buyerEmail - коректний формат e-mail; seatsCount - ціле число від 1 до 10. localStorage: зберігати чернетку форми покупки квитків.
16. Дошка оголошень	Форма: нове оголошення - title, price, category (select), phone. Валідація: phone - формат телефонного номера (preg_match); price - число не менше 0. localStorage: запам'ятати обрану категорію оголошень для фільтра списку.
17. Трекер звичок (habit tracker)	Форма: нова звичка - name, targetPerWeek. Валідація: targetPerWeek - ціле число від 1 до 7; name обов'язкове. localStorage: зберігати відмітки виконання звички за сьогодні до підтвердження на сервері.
18. Каталог догляду за рослинами (садівництво)	Форма: додавання рослини - name, wateringIntervalDays, lastWatered. Валідація: wateringIntervalDays - ціле число більше 0; lastWatered - коректна дата, не пізніша за сьогодні. localStorage: запам'ятати обране сортування списку рослин (за назвою/за датою поливу).

19. Система тікетів служби підтримки (helpdesk)	Форма: новий тікет - subject, description, priority (select). Валідація: description - не менше 15 символів; priority - одне з дозволених значень списку. localStorage: зберігати чернетку опису тікета до відправлення.
20. Спортивна ліга (турнірна таблиця, матчі, команди)	Форма: результат матчу - teamHome, teamAway, scoreHome, scoreAway. Валідація: scoreHome і scoreAway - цілі числа не менше 0; teamHome не збігається з teamAway. localStorage: запам'ятати останній обраний тур/лігу у фільтрі результатів.